

Personal + Business Finance Manager

Complete Flutter Architecture, App Flow and AI Prompt Guide

1. Main App Concept

App-এর temporary name:

Smart Hisab

App-এর ভিতরে দুই ধরনের workspace থাকবে:

```
User Account
├── Personal Workspace
├── My Shop
├── Freelance Business
└── Second Business
```

একজন user চাইলে:

- Personal income-expense track করবে।
- একটি বা একাধিক business manage করবে।
- প্রতিটি workspace-এর accounts, transactions এবং reports আলাদা থাকবে।

A workspace is an isolated financial area.

বাংলা: Workspace হলো আলাদা হিসাবের জায়গা, যেখানে অন্য workspace-এর data mix হবে না।

2. Recommended Technology Stack

Flutter Frontend

```
Flutter
Dart
Riverpod
GoRouter
Drift / SQLite
Freezed
```

JSON Serializable
Dio
Flutter Secure Storage
Local Auth
FL Chart
Firebase Crashlytics
Firebase Analytics

Backend

আপনার Node.js experience থাকায়:

Node.js
Express.js or NestJS
TypeScript
PostgreSQL
Prisma ORM
JWT Authentication
Redis optional
Cloud Storage

Recommended Database

Local database:

Drift + SQLite

Server database:

PostgreSQL

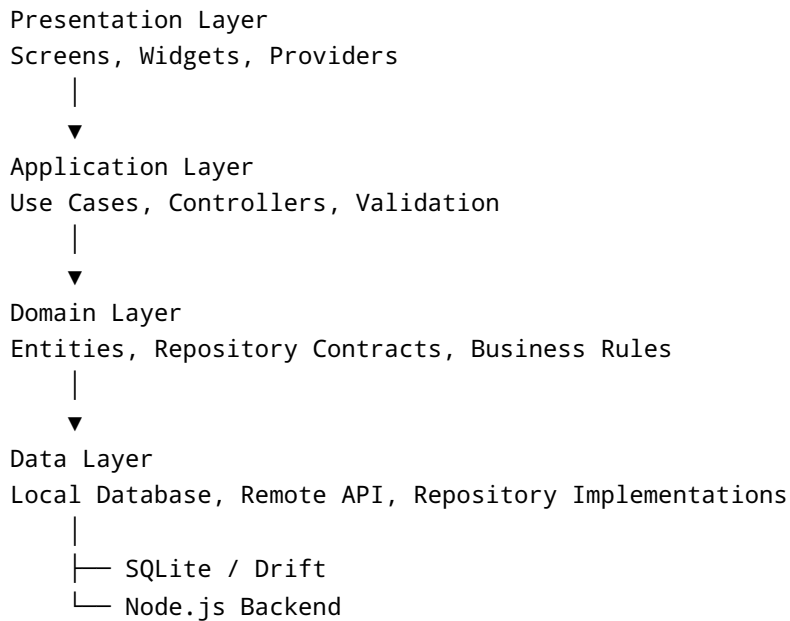
Financial app-এর জন্য relational database ভালো, কারণ:

- Account-এর সঙ্গে transaction সম্পর্কিত।
- Transaction-এর সঙ্গে category সম্পর্কিত।
- Invoice-এর সঙ্গে customer এবং items সম্পর্কিত।
- Business-এর সঙ্গে user এবং employees সম্পর্কিত।

3. High-Level Architecture

Flutter UI





Presentation Layer

এখানে থাকবে:

- Screens
- Widgets
- Dialogs
- Riverpod providers
- Form state
- Navigation

Application Layer

এখানে থাকবে:

- Add transaction use case
- Transfer money use case
- Create invoice use case
- Receive customer payment use case
- Generate report use case
- Sync data use case

Domain Layer

এখানে app-এর মূল business rules থাকবে।

উদাহরণ:

Expense amount must be greater than zero.
A transfer requires two different accounts.

A customer payment cannot exceed the due amount unless advance payment is supported.

Data Layer

এখানে থাকবে:

- Drift tables
- Local data sources
- API clients
- Repository implementations
- DTO models
- Sync engine

4. Recommended Flutter Folder Structure

```
lib/  
├─ main.dart  
├─ bootstrap.dart  
├─  
├─ app/  
│   ├─ app.dart  
│   ├─ router/  
│   │   ├─ app_router.dart  
│   │   └─ route_names.dart  
│   └─ theme/  
│       ├─ app_theme.dart  
│       ├─ app_colors.dart  
│       └─ app_typography.dart  
├─  
├─ core/  
│   ├─ constants/  
│   ├─ errors/  
│   ├─ extensions/  
│   ├─ network/  
│   ├─ security/  
│   ├─ database/  
│   ├─ localization/  
│   ├─ utils/  
│   └─ widgets/  
├─  
├─ features/  
│   ├─ authentication/  
│   ├─ onboarding/  
│   ├─ workspace/  
│   └─ dashboard/
```

```

├── accounts/
├── categories/
├── transactions/
├── transfers/
├── budgets/
├── customers/
├── suppliers/
├── sales/
├── purchases/
├── invoices/
├── dues/
├── loans/
├── recurring_transactions/
├── reports/
├── reminders/
├── attachments/
├── backup_sync/
├── subscriptions/
├── settings/
└── services/
    ├── notification_service.dart
    ├── biometric_service.dart
    ├── file_service.dart
    ├── export_service.dart
    ├── analytics_service.dart
    └── sync_service.dart

```

প্রতিটি feature-এর ভিতরে:

```

transactions/
├── data/
│   ├── data_sources/
│   ├── dtos/
│   ├── repositories/
│   └── tables/
├── domain/
│   ├── entities/
│   ├── repositories/
│   └── use_cases/
└── presentation/
    ├── providers/
    ├── screens/
    └── widgets/

```

5. Main App Modules

Module 1: Authentication

Features:

- Email login
- Google login
- Apple login
- Guest mode
- Forgot password
- Logout
- Device management

MVP-তে guest/local mode রাখা ভালো।

Continue Offline
Sign In
Create Account

Cloud sync ব্যবহার করতে user-কে login করতে হবে।

Module 2: Workspace

Workspace types:

Personal
Business

Business workspace-এর additional information:

- Business name
- Business type
- Address
- Phone
- Logo
- Currency
- Financial year
- Tax/VAT information
- Invoice settings

Workspace switcher:

Current Workspace: My Shop

Switch to:

- Personal
- Freelance Work
- New Business

Module 3: Accounts

Account types:

Personal

- Cash
- Bank
- Card
- bKash
- Nagad
- Rocket
- Savings
- Investment

Business

- Office Cash
- Bank
- Mobile Wallet
- Petty Cash
- Customer Receivable
- Supplier Payable

Account fields:

Account ID
Workspace ID
Name
Type
Opening Balance
Current Balance
Currency
Icon
Color
Is Active
Created At
Updated At

Module 4: Transactions

Transaction types:

Income
Expense
Transfer
Adjustment

Personal income examples:

- Salary
- Freelance
- Bonus
- Investment income
- Gift

Personal expense examples:

- Food
- Transport
- Rent
- Shopping
- Medical
- Education

Business income examples:

- Product sales
- Service sales
- Customer payment
- Commission
- Other income

Business expense examples:

- Purchase
- Rent
- Salary
- Electricity
- Internet
- Marketing
- Transport
- Maintenance

Transaction fields:

Transaction ID
Workspace ID
Account ID
Category ID
Type
Amount
Date

Note
Reference Number
Contact ID
Attachment ID
Created At
Updated At
Sync Status

6. Important Accounting Architecture

Personal finance এবং business finance-এর হিসাব একইভাবে handle করা উচিত নয়।

Personal Mode

Personal mode-এ simple cash-flow model ব্যবহার করা যাবে:

Income increases account balance.
Expense decreases account balance.
Transfer moves money between accounts.

Business Mode

Business mode-এ ভবিষ্যতের জন্য double-entry accounting structure রাখা ভালো।

Every business transaction affects at least two ledger accounts.

বাংলা: প্রতিটি business transaction কমপক্ষে দুটি ledger account-কে পরিবর্তন করে।

উদাহরণ: Cash sale ₳5,000

Cash Account	+5,000
Sales Income	+5,000

Business expense ₳1,000:

Office Expense	+1,000
Cash Account	-1,000

এই জন্য backend/database-এ structure হতে পারে:

Transaction

└ Transaction Line 1
└ Transaction Line 2

Transaction Line Model

Transaction Line ID
Transaction ID
Ledger Account ID
Debit Amount
Credit Amount

MVP-তে UI simple থাকবে, কিন্তু database structure advanced রাখা যাবে।

7. Customer and Supplier Module

Customer

Fields:

Name
Phone
Email
Address
Opening Due
Credit Limit
Notes

Customer details screen:

Customer Name
Total Sales
Total Paid
Current Due
Transaction History
Invoices
Payments

Supplier

Fields:

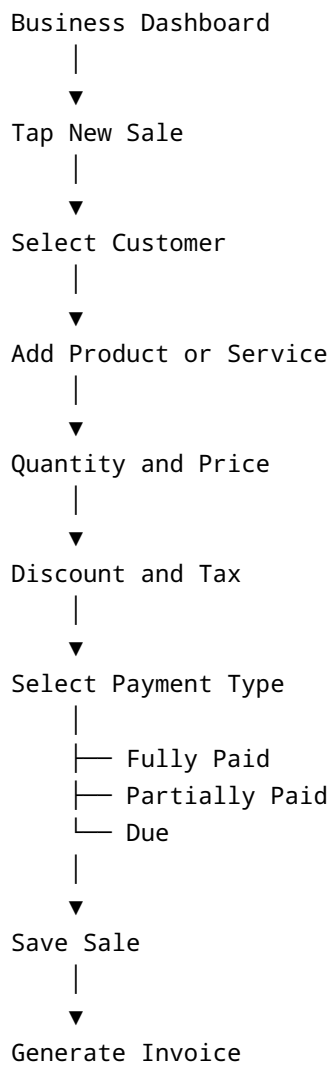
Name
Phone

Address
Opening Payable
Notes

Supplier details screen:

Total Purchases
Total Paid
Current Payable
Purchase History
Payments

8. Sales Flow



Sale Result

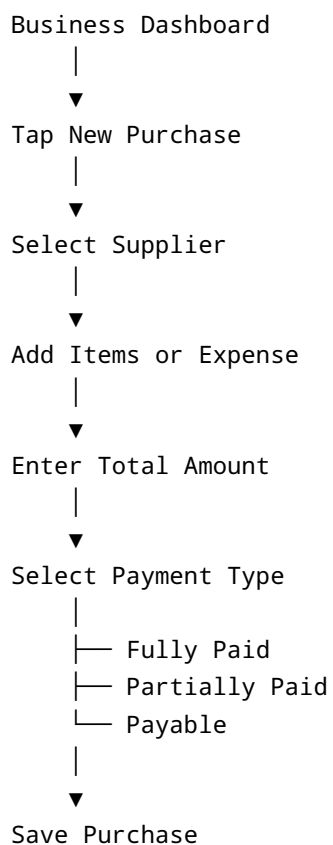
Fully paid sale:

Sales increases
Cash/Bank increases
Customer due remains zero

Partially paid sale:

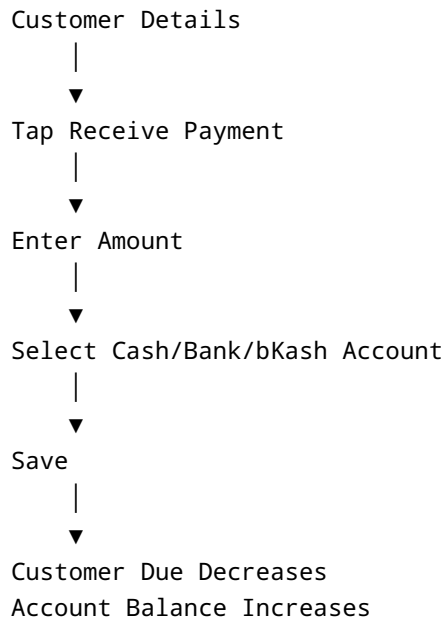
Sales increases
Cash increases by paid amount
Customer due increases by remaining amount

9. Purchase Flow

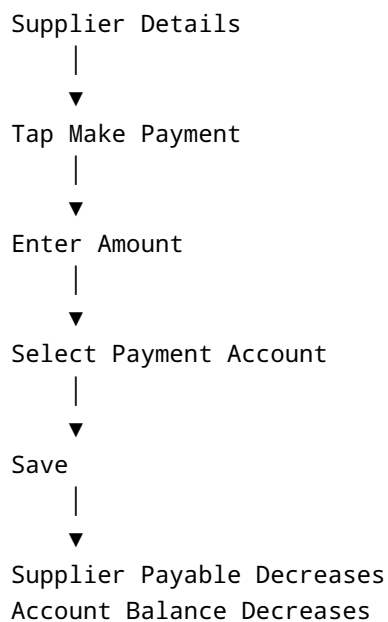


10. Due Management Flow

Customer Due Collection



Supplier Payment



11. Budget Module

Budget types:

- Monthly overall budget
- Category budget
- Account spending limit
- Business expense budget

Example:

Food Budget: ₪8,000
Used: ₪6,500
Remaining: ₪1,500

Budget status:

Safe: 0-70%
Warning: 71-90%
Exceeded: Above 90% or 100%

12. Loan Module

Loan types:

I Borrowed
I Lent
Business Loan
Customer Loan
Employee Advance

Fields:

Person
Principal Amount
Interest optional
Start Date
Due Date
Installment Amount
Paid Amount
Remaining Amount
Reminder

13. Reports Module

Personal Reports

- Monthly income vs expense
- Expense by category
- Account balances
- Budget performance
- Savings trend
- Cash flow
- Loan summary

Business Reports

- Daily sales
- Monthly sales
- Purchase report
- Profit and loss
- Customer due report
- Supplier payable report
- Cash flow report
- Account statement
- Expense by category
- Invoice report

Advanced reports:

- Balance sheet
- Trial balance
- General ledger
- Tax/VAT report

MVP-তে advanced accounting reports না রাখলেও হবে।

14. Dashboard Flow

Personal Dashboard

Workspace Selector

Total Balance

This Month

Income | Expense | Savings

Budget Progress

Expense Chart

Recent Transactions

Quick Actions:

Add Income

Add Expense

Transfer

Business Dashboard

Workspace Selector

Today's Sales

Today's Expense

Today's Profit

Cash and Bank Balance

Customer Due

Supplier Payable

Recent Sales

Quick Actions:

New Sale

New Expense

Receive Payment

Create Invoice

15. Main Navigation

Recommended bottom navigation:

Home

Transactions

Add

Reports

More

Center Add button options:

Personal Workspace:

- Add Income

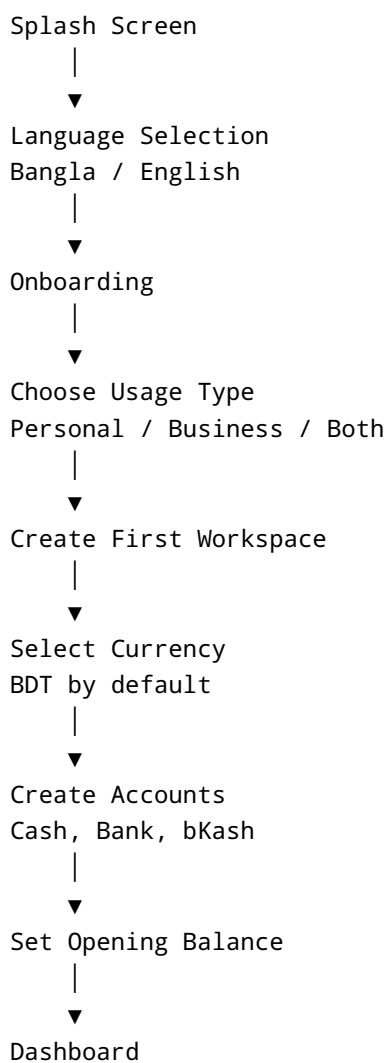
- Add Expense

- Transfer

Business Workspace:

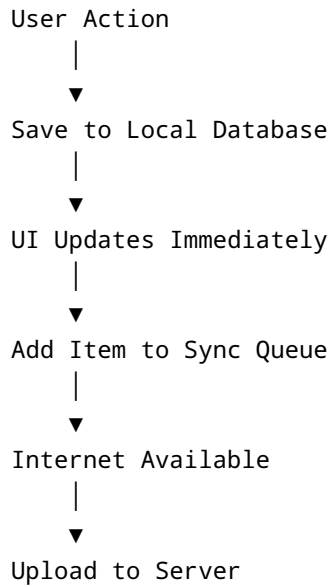
- New Sale
- New Purchase
- Add Expense
- Receive Payment
- Make Payment

16. Complete First-Time User Flow



17. Local-First and Cloud Sync Architecture

App internet ছাড়াও কাজ করতে হবে।



Local-first means the app saves data on the device before sending it to the server.

বাংলা: Local-first মানে আগে device-এ data save হবে, পরে server-এ sync হবে।

প্রতিটি syncable table-এ রাখুন:

```
id
createdAt
updatedAt
deletedAt
version
syncStatus
deviceId
```

Sync status:

```
pending
syncing
synced
failed
```

Conflict resolution:

```
Latest Updated Record Wins
```

Financial data-এর জন্য পরে version checking এবং audit log যোগ করুন।

18. Security Architecture

Required features:

- PIN lock
- Biometric lock
- Secure token storage
- Encrypted API communication
- Automatic session timeout
- Screenshot blocking optional
- Local backup encryption
- Audit log
- Delete account
- Export user data

Sensitive values যেমন password বা token `SharedPreferences` -এ রাখবেন না।

Use:

```
flutter_secure_storage  
Android Keystore  
iOS Keychain
```

19. Android Native Features

Flutter দিয়েই বেশিরভাগ feature করা যাবে।

Android Native Kotlin প্রয়োজন হতে পারে:

- Home-screen widget
- Advanced scheduled alarms
- Notification actions
- Background sync
- Bluetooth thermal printer
- SMS transaction reading
- Advanced file access

MVP-তে SMS reading বাদ দিন। Permission এবং store policy complex হতে পারে।

20. Database Entities

Minimum entities:

User
Workspace
BusinessProfile
Account
Category
Transaction
TransactionLine
Contact
Customer
Supplier
Sale
SaleItem
Purchase
PurchaseItem
Invoice
InvoiceItem
Payment
Budget
Loan
RecurringTransaction
Attachment
Reminder
SyncQueue
AuditLog
Subscription

Important Relations

User
└─ has many Workspaces

Workspace
└─ has many Accounts
└─ has many Categories
└─ has many Transactions
└─ has many Customers
└─ has many Suppliers
└─ has many Budgets

Sale
└─ belongs to Customer
└─ has many SaleItems
└─ may create Invoice

Transaction
└─ has many TransactionLines

21. Backend API Structure

```
/api/v1/auth  
/api/v1/users  
/api/v1/workspaces  
/api/v1/accounts  
/api/v1/categories  
/api/v1/transactions  
/api/v1/transfers  
/api/v1/customers  
/api/v1/suppliers  
/api/v1/sales  
/api/v1/purchases  
/api/v1/invoices  
/api/v1/payments  
/api/v1/budgets  
/api/v1/loans  
/api/v1/reports  
/api/v1/sync  
/api/v1/subscriptions
```

Example:

```
POST   /transactions  
GET    /transactions  
GET    /transactions/:id  
PATCH /transactions/:id  
DELETE /transactions/:id
```

Sync endpoint:

```
POST /sync/push  
GET  /sync/pull?lastSyncedAt=...
```

22. MVP Scope

প্রথম version-এ শুধু এগুলো রাখুন:

Personal

- Workspace
- Accounts
- Income
- Expense

- Transfer
- Categories
- Monthly budget
- Basic reports
- Local backup
- PIN/biometric

Business

- Business workspace
- Customers
- Suppliers
- Sales
- Purchases
- General expenses
- Customer due
- Supplier payable
- Basic invoice
- Profit summary

MVP-তে বাদ দিন:

- Inventory
- Employee management
- Full accounting reports
- SMS reading
- Team collaboration
- Multi-device realtime sync
- Complex tax calculation
- POS

23. Development Phases

Phase 1: Foundation

- Project setup
- Architecture
- Theme
- Localization
- Database
- Navigation
- Workspace

Phase 2: Personal Finance

- Accounts
- Categories
- Income
- Expense

- Transfer
- Dashboard
- Budget
- Reports

Phase 3: Business Finance

- Customers
- Suppliers
- Sales
- Purchases
- Due
- Payments
- Profit summary

Phase 4: Security and Backup

- PIN
- Biometric
- Export
- Import
- Cloud sync

Phase 5: Commercial Features

- Subscription
- PDF invoice
- Premium reports
- Cloud backup
- Shared business
- Analytics

24. AI Prompt Rules

AI-কে পুরো app একসঙ্গে বানাতে বলবেন না।

এই rules সব prompt-এর শুরুতে ব্যবহার করুন:

This is an existing Flutter finance application.

Act as a senior Flutter and financial software architect.

Important rules:

- Do not rewrite the whole project.
- Work feature by feature and file by file.
- Use clean architecture.
- Use Riverpod for state management.
- Use Drift for the local database.

- Keep domain logic independent from Flutter UI.
- Use immutable models where practical.
- Add validation and error handling.
- Do not create unnecessary abstractions.
- Do not change existing public APIs without explaining why.
- All code comments must be in English.
- Wait for my existing files before modifying them.

25. Prompt 1: Architecture

I am building a Personal and Business Finance Manager using Flutter and Dart.

The app will support multiple workspaces:

- Personal workspace
- Business workspace
- Multiple businesses

Design the complete clean architecture.

Include:

- Feature-based folder structure
- Presentation, domain and data layers
- Riverpod state management
- Drift local database
- Repository pattern
- Offline-first sync
- Security layer
- Error handling

Do not write implementation code yet.

26. Prompt 2: Database Design

Design the Drift database schema for my Flutter Personal and Business Finance Manager.

Include these entities:

- Workspace
- Account
- Category
- Transaction
- TransactionLine
- Customer
- Supplier

- Sale
- SaleItem
- Purchase
- PurchaseItem
- Payment
- Budget
- Loan
- Attachment
- SyncQueue

Requirements:

- Use UUID string IDs
- Support soft deletion
- Support offline synchronization
- Store createdAt and updatedAt
- Add indexes where useful
- Explain all relationships
- Do not create UI code

27. Prompt 3: Workspace Feature

Create the workspace feature for my existing Flutter project.

Requirements:

- Personal and business workspace types
- User can create multiple workspaces
- User can switch current workspace
- All financial data must be filtered by workspaceId
- Store selected workspace locally
- Use Riverpod and Drift
- Create one file at a time

28. Prompt 4: Account Module

Create the account management domain layer.

Account types:

- Cash
- Bank
- Card
- Mobile wallet
- Savings
- Investment
- Business cash

- Petty cash

Requirements:

- Create account
- Update account
- Archive account
- Calculate account balance
- Support opening balance
- Keep balance calculation in domain logic
- No UI code yet

29. Prompt 5: Transaction Logic

Create the transaction domain logic for my Flutter finance app.

Transaction types:

- Income
- Expense
- Transfer
- Adjustment

Requirements:

- Income increases account balance
- Expense decreases account balance
- Transfer decreases source and increases destination
- Prevent transfer to the same account
- Validate positive amounts
- Support notes, categories, dates and attachments
- Use database transactions to ensure consistency
- Pure Dart domain logic where possible

30. Prompt 6: Personal Dashboard

Create a responsive Personal Finance Dashboard using Flutter.

Show:

- Total balance
- Monthly income
- Monthly expense
- Monthly savings
- Budget progress
- Expense by category chart
- Recent transactions
- Quick action buttons

Requirements:

- Use Riverpod
- Read data from existing repositories
- Support Bangla and English
- Support light and dark themes
- Do not include business data

31. Prompt 7: Business Module

Design the business finance module for my existing Flutter app.

Include:

- Customers
- Suppliers
- Sales
- Purchases
- Business expenses
- Customer dues
- Supplier payables
- Payments
- Basic profit calculation

Explain:

- Domain entities
- Use cases
- Database relationships
- Provider structure
- Screen flow

Do not write code yet.

32. Prompt 8: Sales and Due Logic

Create the sales and customer due domain logic.

Requirements:

- Support fully paid sale
- Support partially paid sale
- Support fully due sale
- Update customer balance correctly
- Update receiving account balance
- Create payment history
- Prevent negative payments

- Use atomic database transactions
- Return clear success or failure results

Atomic transaction means either all related database changes succeed, or none of them are saved.

বাংলা: Atomic transaction মানে সব related change সফল হবে, না হলে কোনো change-ই save হবে না।

33. Prompt 9: Reports

Create the report domain layer for my Flutter finance app.

Personal reports:

- Income vs expense
- Expense by category
- Account balances
- Budget performance

Business reports:

- Sales report
- Purchase report
- Expense report
- Profit summary
- Customer due
- Supplier payable
- Cash flow

Requirements:

- Support daily, weekly, monthly and custom date ranges
- Use efficient Drift queries
- Keep report calculations out of UI widgets

34. Prompt 10: Backup and Restore

Create a local backup and restore system.

Requirements:

- Export workspace data to JSON
- Include database schema version
- Support encrypted backup file
- Validate backup before importing
- Prevent duplicate records
- Allow user to choose a specific workspace

- Show import progress
- Keep existing data safe if import fails

35. Prompt 11: Cloud Sync

Design an offline-first synchronization system between Flutter Drift and a Node.js PostgreSQL backend.

Requirements:

- Push local pending changes
- Pull server changes after last sync time
- Support soft deletion
- Support record versions
- Handle conflicts
- Maintain a sync queue
- Retry failed requests
- Avoid duplicate transactions
- Explain API request and response formats

Do not write the entire implementation at once.

36. Prompt 12: Security Review

Review the security architecture of my Flutter finance app.

Check:

- Authentication token storage
- Biometric lock
- PIN security
- Local database protection
- Backup encryption
- API security
- Input validation
- Sensitive log prevention
- Screenshot protection
- Account deletion
- Privacy settings

Provide a prioritized checklist.

Do not rewrite the project.

37. Recommended Development Order

1. Project architecture
2. Drift database
3. Workspace
4. Accounts
5. Categories
6. Income and expense
7. Transfer
8. Personal dashboard
9. Budget
10. Personal reports
11. Customer and supplier
12. Sales and purchases
13. Due and payments
14. Business dashboard
15. Invoice
16. Backup
17. Security
18. Cloud sync
19. Subscription
20. Store release

38. Final Product Positioning

App-এর marketing message হতে পারে:

Manage your personal money and small business accounts in one secure app.

বাংলা:

ব্যক্তিগত এবং ছোট ব্যবসার সব হিসাব এক নিরাপদ app-এ পরিচালনা করুন।

Target users:

- চাকরিজীবী
- Freelancer
- Student
- Family
- Small shop owner
- Online seller
- Service provider
- Small agency
- Home business owner

সব feature শুরুতে তৈরি করবেন না। প্রথমে stable MVP release করুন, তারপর user feedback অনুযায়ী sales, inventory, invoice এবং cloud collaboration উন্নত করুন।